

**LAPORAN PRAKTIKUM
STRUKTUR DATA**

“Algoritma Sorting”

disusun Oleh:

Muhammad Fedora Argadyaksa

2511533016

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T

Asisten Praktikum:

Sherly Sukmadira



**DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS**

2026

KATA PENGANTAR

Assalamu'alaikum Warahmatullahi Wabarakatuh

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum *Struktur Data* ini dengan baik dan tepat waktu.

Laporan ini disusun sebagai salah satu syarat untuk memenuhi tugas praktikum pada mata kuliah Struktur Data. Dalam laporan ini, penulis memaparkan hasil praktikum yang telah dilakukan, mulai dari konsep dasar hingga implementasi struktur data seperti array, stack, queue, dan linked list dalam pemrograman.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat banyak kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun dari pembaca demi perbaikan di masa yang akan datang.

Tidak lupa, penulis mengucapkan terima kasih kepada dosen pengampu dan semua pihak yang telah membantu dalam pelaksanaan praktikum serta penyusunan laporan ini.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat dan menambah wawasan bagi pembaca, khususnya dalam memahami konsep dan penerapan struktur data.

Padang, 21 Mei 2025

M. Fedora Argadyaksa

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Tujuan.....	1
1.3 Manfaat.....	2
BAB II PEMBAHASAN.....	3
2.1 Pengertian Algoritma Sorting.....	3
2.2 Macam-Macam Algoritma Sorting yang dipelajari pada praktikum	3
2.3 Implementasi Stack pada Java.....	4
2.2.1 Class Bubble sort (BubbleSort_2511533016)	4
2.2.2 Class Selection sort (SelectionSort_2511533016)	6
2.2.3 Class Insertion sort (InsertionSort_2511533016).....	8
2.2.4 Class Insertion Sort GUI (insertionSortGUI_2511533016)....	10
BAB III PENUTUP	13
3.1 Kesimpulan.....	13
3.2 Saran	13
DAFTAR PUSTAKA.....	14
LAPORAN TUGAS PRAKTIKUM	15
1. Class Mahasiswa (Mahasiswa_2511533016).....	15
2. Class Sorting GUI (SortingGUI_2511533016).....	17

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman dan pengolahan data, salah satu operasi yang paling sering dilakukan adalah mengurutkan sekumpulan data agar lebih mudah dikelola, dicari, maupun dianalisis. Kebutuhan ini muncul di hampir semua bidang, mulai dari sistem perbankan yang mengurutkan transaksi berdasarkan tanggal, aplikasi e-commerce yang menampilkan produk dari harga terendah ke tertinggi, hingga sistem akademik yang mengurutkan nilai mahasiswa. Tanpa adanya mekanisme pengurutan yang baik, proses pencarian dan pengelolaan data dalam jumlah besar akan menjadi sangat lambat dan tidak efisien.

Algoritma sorting atau algoritma pengurutan hadir sebagai solusi dari permasalahan tersebut. Terdapat berbagai macam algoritma sorting yang telah dikembangkan, masing-masing memiliki cara kerja, kelebihan, dan kekurangannya sendiri. Beberapa yang umum dipelajari antara lain Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, dan Quick Sort. Setiap algoritma memiliki karakteristik yang berbeda dalam hal kompleksitas waktu dan ruang, sehingga pemilihan algoritma yang tepat sangat bergantung pada kondisi dan kebutuhan data yang akan diurutkan. Oleh karena itu, pemahaman mendalam tentang algoritma sorting menjadi hal yang sangat penting bagi setiap mahasiswa yang mempelajari ilmu komputer dan struktur data.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum pekan 7 tentang Sorting adalah sebagai berikut:

1. Memahami konsep dasar dan cara kerja dari berbagai algoritma sorting yang umum digunakan dalam pemrograman.
2. Mampu mengimplementasikan algoritma sorting ke dalam kode program menggunakan bahasa pemrograman Java.
3. Melatih kemampuan berpikir logis dan analitis dalam memilih algoritma

4. sorting yang paling sesuai dengan kebutuhan dan karakteristik data yang ada.

1.3 Manfaat

Manfaat yang diharapkan dari pelaksanaan praktikum pekan 7 tentang Sorting adalah sebagai berikut:

1. Meningkatkan kemampuan dalam menulis program yang lebih efisien dan optimal dalam mengelola serta memproses data dalam jumlah besar.
2. Memberikan pemahaman yang lebih baik tentang bagaimana sebuah sistem komputer bekerja dalam mengurutkan dan mengorganisir data secara terstruktur.
3. Menjadi bekal penting dalam menghadapi permasalahan nyata di dunia kerja yang seringkali melibatkan pengolahan data dalam skala besar seperti di bidang teknologi, keuangan, maupun kesehatan.

BAB II

PEMBAHASAN

2.1 Pengertian Algoritma Sorting

Sorting atau pengurutan adalah proses menyusun sekumpulan data ke dalam urutan tertentu, baik dari nilai terkecil ke terbesar (ascending) maupun dari nilai terbesar ke terkecil (descending). Dalam ilmu komputer, sorting merupakan salah satu operasi fundamental yang sangat sering digunakan karena hampir semua proses pengolahan data membutuhkan data yang terurut agar lebih mudah diakses, dicari, dan dianalisis. Algoritma sorting sendiri adalah langkah-langkah sistematis yang digunakan oleh program untuk melakukan proses pengurutan tersebut secara otomatis dan efisien.

2.2 Macam-Macam Algoritma Sorting yang dipelajari pada praktikum

1. Bubble Sort

Bubble Sort adalah algoritma pengurutan paling sederhana yang bekerja dengan cara membandingkan dua elemen yang berdekatan secara berulang, kemudian menukarnya jika urutannya salah. Proses ini terus diulang hingga tidak ada lagi pertukaran yang perlu dilakukan. Dinamakan Bubble Sort karena elemen dengan nilai terbesar akan "menggelembung" ke posisi paling akhir seperti gelembung yang naik ke permukaan air. Algoritma ini mudah dipahami namun kurang efisien untuk data dalam jumlah besar karena kompleksitas waktunya $O(n^2)$.

2. Selection Sort

Selection Sort bekerja dengan cara mencari elemen terkecil dari seluruh data yang belum terurut, kemudian menukarnya dengan elemen di posisi paling depan. Proses ini diulangi terus menerus hingga semua elemen berada di posisi yang benar. Keunggulan Selection Sort adalah jumlah pertukaran yang dilakukan lebih sedikit dibanding Bubble Sort, namun kompleksitas waktunya tetap $O(n^2)$ sehingga masih kurang efisien untuk data berukuran besar.

3. Insertion Sort

Insertion Sort bekerja dengan cara mengambil satu elemen dari data yang belum terurut, kemudian menyisipkannya ke posisi yang tepat di antara elemen-elemen yang sudah terurut sebelumnya. Cara kerjanya mirip dengan bagaimana seseorang mengurutkan kartu di tangannya satu per satu. Algoritma ini sangat efisien untuk data yang sudah hampir terurut dan cocok digunakan pada data berukuran kecil, dengan kompleksitas waktu terbaik $O(n)$ dan terburuk $O(n^2)$.

2.3 Implementasi Stack pada Java

Pada praktikum pekan 7 mahasiswa disuruh untuk mengikuti intruksi dari dosen. Yaitu menuliskan kembali kode program yang sudah ada di layar TV. Kode program yang dibuat tentang Bubble sort, Selection sort, Insertion sort di bawah ini kode program yang dituliskan pada praktikum pekan 7.

2.2.1 Class Bubble sort (BubbleSort_2511533016)

```

4. package Pekan7_2511533016;
5.
6. public class BubleSort_2511533016 {
7.     public static void bubbleSort_2511533016(int[] arr) {
8.         int n_3016 = arr.length;
9.         for (int i_3016 = 0; i_3016 < n_3016; i_3016++) {
10.             for (int j_3016 = 0; j_3016 < n_3016 - i_3016
11. - 1; j_3016++) {
12.                 if(arr[j_3016] > arr [j_3016 + 1]) {
13.                     int temp_3016 = arr[j_3016];
14.                     arr[j_3016] = arr[j_3016 + 1];
15.                     arr[j_3016 + 1] = temp_3016;
16.                     System.out.println("data:" +
17. arr[j_3016]+" "+arr[j_3016+1]);
18.                 }
19.             }
20.         }
21.     }
22.     public static void main(String[] args) {
23.         int arr[] = { 23, 78, 45, 8, 32, 56, 1};
24.         int n_3016 = arr.length;
25.         System.out.print("array yang belum terurut : ");
26.         for(int i_3016 = 0; i_3016 < n_3016; i_3016++)
27.             System.out.print(arr[i_3016] + " ");
28.         System.out.println("");
29.         bubbleSort_2511533016(arr);
30.         System.out.print("array yang terurut menggunakan
31. BubbleSort : ");
32.         for (int i_3016 = 0; i_3016 < n_3016; i_3016++)

```

```

30.         System.out.print(arr[i_3016] + " ");
31.         System.out.println("");
32.     }
33. }

array yang belum terurut : 23 78 45 8 32 56 1
data:45 78
data:8 78
data:32 78
data:56 78
data:1 78
data:8 45
data:32 45
data:1 56
data:8 23
data:1 45
data:1 32
data:1 23
data:1 8
array yang terurut menggunakan BubbleSort : 1 8 23 32 45 56 78

```

Gambar 2.2.1

Kode di atas adalah implementasi algoritma Bubble Sort yang berada dalam package `Pekan7_2511533016`. Kelas utamanya bernama `BubleSort_2511533016` yang berisi dua bagian utama yaitu fungsi untuk melakukan proses pengurutan dan fungsi main untuk menjalankan program.

Fungsi `bubbleSort_2511533016` menerima parameter berupa array integer yang akan diurutkan. Di dalamnya terdapat dua perulangan `for` yang bersarang. Perulangan luar berjalan sebanyak jumlah elemen array untuk mengontrol berapa kali proses perbandingan dilakukan secara keseluruhan. Perulangan dalam bertugas membandingkan setiap dua elemen yang berdekatan yaitu `arr[j_3016]` dan `arr[j_3016 + 1]`. Jika elemen di posisi kiri lebih besar dari elemen di posisi kanan, maka kedua elemen tersebut ditukar posisinya menggunakan variabel sementara `temp_3016`. Setiap kali pertukaran terjadi, program juga mencetak data yang baru saja ditukar agar proses pengurutan bisa terlihat secara bertahap. Batas perulangan dalam dikurangi dengan nilai `i_3016` karena setiap kali perulangan luar selesai, elemen terbesar sudah dipastikan berada di posisi paling akhir sehingga tidak perlu dibandingkan lagi.

Pada fungsi main, program mendefinisikan array dengan tujuh elemen yaitu {23, 78, 45, 8, 32, 56, 1} yang belum terurut. Sebelum proses pengurutan, program mencetak isi array awal terlebih dahulu, kemudian memanggil fungsi `bubbleSort_2511533016` untuk mengurutkan array tersebut. Setelah proses selesai, program mencetak kembali isi array yang kini sudah terurut dari nilai terkecil ke terbesar yaitu 1 8 23 32 45 56 78.

Kesimpulan : Kode program ini merupakan implementasi algoritma Bubble Sort dalam bahasa Java, yang mendemonstrasikan cara mengurutkan sekumpulan data integer dari nilai terkecil ke terbesar (ascending) dengan cara membandingkan dan menukar dua elemen yang berdekatan secara berulang hingga seluruh data berada dalam urutan yang benar.

2.2.2 Class Selection sort (SelectionSort_2511533016)

```

1. package Pekan7_2511533016;
2.
3. public class SeletionSort_2511533016 {
4.     public static void selectionSort_2511533016(int[] arr) {
5.         int n_3016 = arr.length;
6.         for (int i_3016 = 0; i_3016 < n_3016; i_3016++) {
7.             int minIndex_3016 = i_3016;
8.             for (int j_3016 = i_3016 + 1; j_3016 <
n_3016; j_3016++) {
9.                 if (arr[j_3016] < arr[minIndex_3016]) {
10.                     minIndex_3016 = j_3016;
11.                 }
12.             }
13.             int temp_3016 = arr[i_3016];
14.             arr[i_3016] = arr[minIndex_3016];
15.             arr[minIndex_3016] = temp_3016;
16.         }
17.     }
18.     public static void main(String[] args) {
19.         int arr[] = {23, 78, 45, 8, 32, 56, 1 };
20.         int n_3016 = arr.length;
21.         System.out.printf("array yang belum terurut : \n");
22.         for (int i_3016 = 0; i_3016 < n_3016; i_3016++)
23.             System.out.print(arr[i_3016] + " ");
24.         System.out.println(" ");
25.         selectionSort_2511533016(arr);
26.         System.out.printf("array sudah terurut : \n");
27.         for (int i_3016 = 0; i_3016 < n_3016; i_3016++)
28.             System.out.print(arr[i_3016] + " ");
29.         System.out.println("");
30.     }
31. }

```

```

array yang belum terurut :
23 78 45 8 32 56 1
array sudah terurut :
1 8 23 32 45 56 78

```

Gambar 2.2.2

Kode di atas adalah implementasi algoritma Selection Sort yang berada dalam package `Pekan7_2511533016`. Kelas utamanya bernama `SelelctionSort_2511533016` yang berisi dua bagian utama yaitu fungsi untuk melakukan proses pengurutan dan fungsi main untuk menjalankan program.

Fungsi `selectionSort_2511533016` menerima parameter berupa array integer yang akan diurutkan. Di dalamnya terdapat dua perulangan `for` yang bersarang. Perulangan luar berjalan dari indeks pertama hingga terakhir dan bertugas menentukan posisi mana yang sedang dicari nilai terkecilnya. Pada setiap iterasi perulangan luar, variabel `minIndex_3016` diinisialisasi dengan nilai indeks saat ini sebagai asumsi awal bahwa elemen di posisi tersebut adalah yang terkecil. Perulangan dalam kemudian berjalan mulai dari indeks setelahnya untuk membandingkan setiap elemen dengan elemen yang saat ini dianggap terkecil. Jika ditemukan elemen yang lebih kecil, maka `minIndex_3016` diperbarui ke indeks elemen tersebut. Setelah perulangan dalam selesai dan nilai terkecil sudah ditemukan, program menukar elemen di posisi perulangan luar dengan elemen terkecil yang ditemukan menggunakan variabel sementara `temp_3016`. Proses ini terus berulang hingga seluruh elemen berada di posisi yang benar.

Pada fungsi main, program mendefinisikan array dengan tujuh elemen yaitu `{23, 78, 45, 8, 32, 56, 1}` yang belum terurut. Sebelum proses pengurutan, program mencetak isi array awal terlebih dahulu menggunakan `printf`, kemudian memanggil fungsi `selectionSort_2511533016` untuk mengurutkan array tersebut. Setelah proses selesai, program mencetak kembali isi array yang kini sudah terurut dari nilai terkecil ke terbesar yaitu `1 8 23 32 45 56 78`.

Kesimpulan: Kode program ini merupakan implementasi algoritma Selection Sort dalam bahasa Java, yang mendemonstrasikan cara mengurutkan sekumpulan data integer dari nilai terkecil ke terbesar (ascending) dengan cara mencari elemen terkecil dari data yang belum terurut kemudian menukarnya ke posisi yang tepat secara berulang hingga seluruh data terurut.

2.2.3 Class Insertion sort (InsertionSort_2511533016)

```

1. package Pekan7_2511533016;
2.
3. public class InsertionSort_2511533016 {
4.     public static void insertionSort_2511533016 (int[] arr) {
5.         int n_3016 = arr.length;
6.         for (int i = 1; i < n_3016; i++) {
7.             int key_3016 = arr[i];
8.             int j_3016 = i - 1;
9.             while (j_3016 >= 0 && arr[j_3016] > key_3016)
10.            {
11.                arr[j_3016 + 1] = arr[j_3016];
12.                j_3016--;
13.            }
14.            arr[j_3016 + 1] = key_3016;
15.        }
16.
17.        public static void main (String[] args) {
18.            int arr[] = { 23, 78, 45, 8, 32, 56, 1 };
19.            int n_3016 = arr.length;
20.            System.out.printf("array yang belum berurutan
21.            : \n");
22.            for (int i_3016 = 0; i_3016 < n_3016;
23.                i_3016++)
24.                System.out.print(arr[i_3016] + " ");
25.            System.out.println("");
26.            insertionSort_2511533016(arr);
27.            System.out.printf("array yang terurut : \n");
28.            for (int i = 0; i < n_3016; i++)
29.                System.out.print(arr[i] + " ");
30.            System.out.println(" ");
31.        }
32.    }

```

```

array yang belum berurutan :
23 78 45 8 32 56 1
array yang terurut :
1 8 23 32 45 56 78

```

Gambar 2.2.3

Kode di atas adalah implementasi algoritma Insertion Sort yang berada dalam package Pekan7_2511533016. Kelas utamanya bernama InsertionSort_2511533016 yang berisi dua bagian utama yaitu fungsi untuk melakukan proses pengurutan dan fungsi main untuk menjalankan program.

Fungsi `insertionSort_2511533016` menerima parameter berupa array integer yang akan diurutkan. Perulangan `for` dimulai dari indeks ke-1 bukan dari indeks ke-0, karena elemen pertama dianggap sudah terurut dengan sendirinya dan tidak perlu dibandingkan. Pada setiap iterasi, elemen yang sedang diproses disimpan ke dalam variabel `key_3016`, kemudian variabel `j_3016` diinisialisasi dengan posisi satu langkah di belakang elemen tersebut. Perulangan `while` kemudian berjalan selama `j_3016` masih valid dan elemen di posisi `j_3016` lebih besar dari `key_3016`. Selama kondisi itu terpenuhi, elemen di posisi `j_3016` digeser satu langkah ke kanan untuk memberi ruang, dan `j_3016` dikurangi satu. Setelah perulangan `while` selesai dan posisi yang tepat sudah ditemukan, `key_3016` disisipkan ke posisi tersebut melalui `arr[j_3016 + 1] = key_3016`. Proses ini terus berulang hingga seluruh elemen berada dalam urutan yang benar.

Pada fungsi `main`, program mendefinisikan array dengan tujuh elemen yaitu `{23, 78, 45, 8, 32, 56, 1}` yang belum terurut. Sebelum proses pengurutan, program mencetak isi array awal terlebih dahulu, kemudian memanggil fungsi `insertionSort_2511533016` untuk mengurutkan array tersebut. Setelah proses selesai, program mencetak kembali isi array yang kini sudah terurut dari nilai terkecil ke terbesar yaitu `1 8 23 32 45 56 78`.

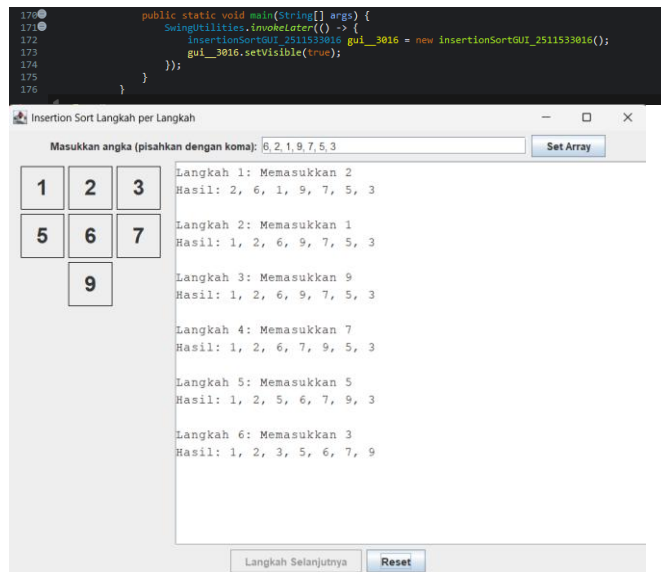
Kesimpulan : Kode program ini merupakan implementasi algoritma Insertion Sort dalam bahasa Java, yang mendemonstrasikan cara mengurutkan sekumpulan data integer dari nilai terkecil ke terbesar (ascending) dengan cara mengambil satu elemen kemudian menyisipkannya ke posisi yang tepat di antara elemen-elemen yang sudah terurut sebelumnya, seperti cara seseorang mengurutkan kartu di tangannya satu per satu.

2.2.4 Class Insertion Sort GUI (insertionSortGUI_2511533016)

```

1 package Pkani7_2511533016;
2
3 import java.awt.BorderLayout;
4 import java.awt.Color;
5 import java.awt.Dimension;
6 import java.awt.EventQueue;
7 import java.awt.FlowLayout;
8 import java.awt.Font;
9
10 import javax.swing.BorderFactory;
11 import javax.swing.JButton;
12 import javax.swing.JFrame;
13 import javax.swing.JLabel;
14 import javax.swing.JOptionPane;
15 import javax.swing.JPanel;
16 import javax.swing.JScrollPane;
17 import javax.swing.JTextArea;
18 import javax.swing.JTextField;
19 import javax.swing.SwingUtilities;
20 import javax.swing.border.InputBorder;
21
22
23 public class InsertionSortGUI_2511533016 extends JFrame {
24     private static final long serialVersionUID = 1L;
25     private int[] array_3016;
26     private JLabel[] labelArray_3016;
27     private JButton stepButton_3016, resetButton_3016, setButton_3016;
28     private JTextField inputField_3016;
29     private JPanel panelArray_3016;
30     private JTextArea stepArea_3016;
31
32     private int i_3016 = 1, j_3016;
33     private boolean sorting_3016 = false;
34     private int stepCount_3016 = 1;
35
36
37     // Create the frame.
38     public InsertionSortGUI_2511533016() {
39         setTitle("Insertion Sort (langkah per langkah)");
40         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
41         setLocationRelativeTo(null);
42         setLayout(new BorderLayout());
43
44         // Panel input
45         JPanel inputPanel = new JPanel(new FlowLayout());
46         inputField_3016 = new JTextField(30);
47         setButton_3016 = new JButton("Set Array");
48         inputPanel.add(new JLabel("Masukkan angka (pisahkan dengan koma):"));
49         inputPanel.add(inputField_3016);
50         inputPanel.add(setButton_3016);
51
52         panelArray_3016 = new JPanel();
53         panelArray_3016.setLayout(new FlowLayout());
54
55         // Panel control
56         JPanel controlPanel = new JPanel();
57         stepButton_3016 = new JButton("Langkah selanjutnya");
58         resetButton_3016 = new JButton("Reset");
59         stepButton_3016.setEnabled(false);
60         controlPanel.add(stepButton_3016);
61         controlPanel.add(resetButton_3016);
62
63         stepArea_3016 = new JTextArea(5, 60);
64         stepArea_3016.setEditable(false);
65         stepArea_3016.setFont(new Font("Monospace", Font.PLAIN, 10));
66         JScrollPane scrollPane = new JScrollPane(stepArea_3016);
67
68         add(inputPanel, BorderLayout.NORTH);
69         add(panelArray_3016, BorderLayout.CENTER);
70         add(controlPanel, BorderLayout.SOUTH);
71         add(scrollPane, BorderLayout.EAST);
72
73         setButton_3016.addActionListener(e -> setArrayFromInput_3016());
74         stepButton_3016.addActionListener(e -> performStep_3016());
75         resetButton_3016.addActionListener(e -> reset_3016());
76
77     }
78
79     private void setArrayFromInput_3016() {
80         String text = inputField_3016.getText().trim();
81         if (text.isEmpty()) return;
82         String[] parts = text.split(",");
83         array_3016 = new int[parts.length];
84         try {
85             for (int k = 0; k < parts.length; k++) {
86                 array_3016[k] = Integer.parseInt(parts[k].trim());
87             }
88         } catch (NumberFormatException e) {
89             JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang dipisahkan "
90                 + "dengan koma", "Error", JOptionPane.ERROR_MESSAGE);
91             return;
92         }
93         i_3016 = 1;
94         stepCount_3016 = 1;
95         sorting_3016 = true;
96         stepButton_3016.setEnabled(true);
97         stepArea_3016.setText("");
98         panelArray_3016.removeAll();
99         labelArray_3016 = new JLabel[array_3016.length];
100         for (int k = 0; k < array_3016.length; k++) {
101             labelArray_3016[k] = new JLabel(String.valueOf(array_3016[k]));
102             labelArray_3016[k].setFont(new Font("Arial", Font.BOLD, 20));
103             labelArray_3016[k].setBorder(BorderFactory.createLineBorder(Color.BLACK));
104             labelArray_3016[k].setPreferredSize(new Dimension(50, 30));
105             labelArray_3016[k].setHorizontalAlignment(SwingConstants.CENTER);
106             panelArray_3016.add(labelArray_3016[k]);
107         }
108         panelArray_3016.revalidate();
109         panelArray_3016.repaint();
110     }
111
112     private void performStep_3016() {
113         if (i_3016 < array_3016.length && sorting_3016) {
114             int key = array_3016[i_3016];
115             j_3016 = i_3016 - 1;
116             StringBuilder stepLog = new StringBuilder();
117             stepLog.append("Langkah ").append(stepCount_3016).append(" : ");
118             stepLog.append("Masukkan ").append(key).append("\n");
119             while (j_3016 >= 0 && array_3016[j_3016] > key) {
120                 array_3016[j_3016 + 1] = array_3016[j_3016];
121                 j_3016--;
122             }
123             array_3016[j_3016 + 1] = key;
124             updateLabels_3016();
125             stepLog.append("Hasil : ").append(arrayToString(array_3016)).append("\n\n");
126             stepArea_3016.append(stepLog.toString());
127             i_3016++;
128             stepCount_3016++;
129             if (i_3016 == array_3016.length) {
130                 sorting_3016 = false;
131                 stepButton_3016.setEnabled(false);
132                 JOptionPane.showMessageDialog(this, "Sorting selesai!");
133             }
134         }
135
136         private void updateLabels_3016() {
137             for (int k = 0; k < array_3016.length; k++) {
138                 labelArray_3016[k].setText(String.valueOf(array_3016[k]));
139             }
140         }
141
142         private void reset_3016() {
143             inputField_3016.setText("");
144             panelArray_3016.removeAll();
145             panelArray_3016.revalidate();
146             panelArray_3016.repaint();
147             stepArea_3016.setText("");
148             stepButton_3016.setEnabled(false);
149             sorting_3016 = false;
150             i_3016 = 1;
151             stepCount_3016 = 1;
152         }
153
154         private String arrayToString(int[] arr_3012) {
155             StringBuilder sb = new StringBuilder();
156             for (int k = 0; k < arr_3012.length; k++) {
157                 sb.append(arr_3012[k]);
158                 if (k < arr_3012.length - 1) sb.append(", ");
159             }
160             return sb.toString();
161         }
162     }
163
164 }

```



Gambar 2.2.4

Kode di atas adalah implementasi Insertion Sort dengan tampilan grafis yang berada dalam package `Pekan7_2511533016`. Kelas utamanya bernama `insertionSortGUI_2511533016` yang mewarisi kelas `JFrame` dari library Java Swing, sehingga program ini memiliki jendela tampilan layaknya aplikasi desktop pada umumnya. Di dalam kelas terdapat beberapa atribut utama seperti array untuk menyimpan data, label untuk menampilkan elemen secara visual, tombol-tombol kontrol, field input, serta area teks untuk menampilkan log setiap langkah pengurutan.

Pada bagian constructor, program membangun tampilan jendela dengan ukuran 750x400 piksel menggunakan layout `BorderLayout`. Terdapat panel input di bagian atas yang berisi field teks untuk memasukkan angka yang dipisahkan koma dan tombol `Set Array`. Di bagian tengah terdapat panel array yang menampilkan elemen-elemen dalam bentuk kotak berlabel. Di bagian bawah terdapat panel kontrol berisi tombol `Langkah Selanjutnya` dan `Reset`, serta di bagian kanan terdapat area teks bercakupan scroll untuk menampilkan log setiap langkah pengurutan.

Fungsi `setArrayFromInput_3016` bertugas membaca input angka dari field teks, memvalidasi formatnya, lalu menginisialisasi array dan menampilkan setiap elemennya sebagai kotak berlabel di panel tengah. Jika input tidak valid atau bukan angka, program akan menampilkan pesan error melalui dialog `JOptionPane`. Setelah array berhasil diset, tombol `Langkah Selanjutnya` akan aktif

dan proses sorting siap dimulai.

Fungsi `performStep_3016` adalah inti dari program ini karena bertugas menjalankan satu langkah Insertion Sort setiap kali tombol Langkah Selanjutnya ditekan. Setiap langkah mengambil elemen key dari posisi `i_3016`, kemudian menggeser elemen-elemen yang lebih besar ke kanan hingga posisi yang tepat ditemukan dan key disisipkan ke sana. Setiap langkah yang terjadi dicatat dalam log dan ditampilkan di area teks sebelah kanan. Setelah semua elemen selesai diurutkan, tombol langkah dinonaktifkan dan program menampilkan pesan bahwa sorting telah selesai.

Fungsi pendukung lainnya adalah `updateLabels_3016` yang memperbarui tampilan kotak-kotak elemen setiap kali terjadi perubahan posisi, `reset_3016` yang mengembalikan seluruh tampilan ke kondisi awal, dan `arrayToString` yang mengubah isi array menjadi teks untuk ditampilkan di log. Pada fungsi main, program dijalankan menggunakan `SwingUtilities.invokeLater` untuk memastikan tampilan GUI dibuat di thread yang benar sesuai standar pemrograman Java Swing.

Kesimpulan: Kode program ini merupakan implementasi **algoritma Insertion Sort berbasis antarmuka grafis (GUI)** menggunakan Java Swing, yang mendemonstrasikan proses pengurutan data secara visual dan interaktif langkah per langkah sehingga pengguna dapat melihat secara langsung bagaimana setiap elemen berpindah posisi selama proses pengurutan berlangsung.

BAB III

PENUTUP

3.1 Kesimpulan

Materi sorting merupakan salah satu konsep dasar dalam struktur data dan algoritma yang digunakan untuk mengurutkan data berdasarkan aturan tertentu, seperti urutan alfabet atau angka. Pada program di atas, proses sorting diterapkan pada data mahasiswa menggunakan tiga metode pengurutan yaitu Insertion Sort, Selection Sort, dan Bubble Sort. Ketiga metode tersebut memiliki cara kerja yang berbeda, namun memiliki tujuan yang sama yaitu menyusun data agar lebih teratur dan mudah dicari maupun diproses.

Program yang dibuat menggunakan Java Swing berhasil menampilkan visualisasi proses sorting secara bertahap melalui antarmuka GUI. Pengguna dapat memasukkan data mahasiswa berupa nama, NIM, dan program studi, kemudian memilih metode sorting yang diinginkan. Setiap langkah pengurutan ditampilkan secara visual sehingga pengguna dapat memahami bagaimana proses perpindahan dan pertukaran data terjadi pada setiap algoritma sorting. Selain itu, program juga menerapkan konsep Object Oriented Programming (OOP) melalui penggunaan class Mahasiswa_2511533016 sebagai model penyimpanan data mahasiswa.

Secara keseluruhan, program ini menunjukkan bahwa visualisasi algoritma dapat membantu dalam memahami konsep sorting dengan lebih mudah, interaktif, dan terstruktur. Program juga membuktikan bahwa Java Swing dapat digunakan untuk membangun aplikasi edukasi sederhana yang mampu menggabungkan konsep GUI, struktur data, algoritma sorting, dan OOP dalam satu sistem.

3.2 Saran

Program dapat dikembangkan dengan menambahkan animasi visual sorting, penggunaan JTable agar tampilan lebih rapi, serta penambahan fitur sorting berdasarkan NIM atau program studi. Selain itu, dapat digunakan algoritma sorting yang lebih cepat seperti Quick Sort atau Merge Sort untuk data yang lebih besar.

DAFTAR PUSTAKA

Astuti, W., & Prabowo, H. (2020). Implementasi Algoritma Sorting pada Struktur Data dalam Bahasa Pemrograman Java. *Jurnal Informatika dan Sistem Informasi*, 6(2), 112–125.

LAPORAN TUGAS

PRAKTIKUM

SOAL :

Program dikembangkan menggunakan Java GUI (Swing) untuk menampilkan proses pengurutan data mahasiswa secara interaktif. Program menyediakan beberapa fitur utama, yaitu:

- Input data mahasiswa secara manual melalui form GUI.
- Menyimpan data mahasiswa ke dalam array atau ArrayList object.
- Pemilihan algoritma sorting menggunakan ComboBox.
- Menampilkan visualisasi proses sorting langkah demi langkah.
- Menampilkan hasil akhir pengurutan nama mahasiswa secara ascending (A-Z).

Perbandingan string dilakukan menggunakan method compareToIgnoreCase() agar proses sorting tidak membedakan huruf kapital dan huruf kecil.

1. Class Mahasiswa (Mahasiswa_2511533016)

```
2. package Pekan7_2511533016;
3.
4. public class Mahasiswa_2511533016 {
5.     // Atribut
6.     private String nama_3016;
7.     private String nim_3016;
8.     private String prodi_3016;
9.
10.    // Constructor
11.    public Mahasiswa_2511533016(String nama_3016, String nim_3016,
    String prodi_3016) {
12.        this.nama_3016 = nama_3016;
13.        this.nim_3016 = nim_3016;
14.        this.prodi_3016 = prodi_3016;
15.    }
16.
17.    // Getter
18.    public String getNama_3016() { return nama_3016; }
19.    public String getNim_3016() { return nim_3016; }
20.    public String getProdi_3016() { return prodi_3016; }
21.
22.    // Setter
23.    public void setNama_3016(String nama_3016) { this.nama_3016 =
    nama_3016; }
24.    public void setNim_3016(String nim_3016) { this.nim_3016 =
    nim_3016; }
25.    public void setProdi_3016(String prodi_3016) { this.prodi_3016 =
    prodi_3016; }
```

```

26.
27.     // toString
28.     @Override
29.     public String toString() {
30.         return nama_3016 + " | " + nim_3016 + " | " + prodi_3016;
31.     }
32. }

```

Gambar 1

Program di atas merupakan class Mahasiswa_2511533016 yang digunakan sebagai representasi atau model data mahasiswa pada aplikasi sorting berbasis Java. Class ini berfungsi untuk menyimpan informasi mahasiswa berupa nama, NIM, dan program studi. Pada bagian awal class terdapat tiga atribut private yaitu nama_3016, nim_3016, dan prodi_3016. Penggunaan access modifier private bertujuan untuk menerapkan konsep enkapsulasi dalam pemrograman berorientasi objek agar data tidak dapat diakses secara langsung dari luar class.

Selanjutnya terdapat constructor Mahasiswa_2511533016() yang digunakan untuk menginisialisasi objek mahasiswa saat pertama kali dibuat. Constructor menerima tiga parameter yaitu nama, NIM, dan program studi, kemudian nilai parameter tersebut disimpan ke dalam atribut class menggunakan keyword this. Dengan adanya constructor, proses pembuatan objek menjadi lebih mudah dan terstruktur karena data mahasiswa langsung diisi saat objek dibuat.

Program juga memiliki beberapa method getter yaitu getNama_3016(), getNim_3016(), dan getProdi_3016() yang digunakan untuk mengambil atau membaca nilai atribut dari luar class. Selain itu terdapat method setter yaitu setNama_3016(), setNim_3016(), dan setProdi_3016() yang berfungsi untuk mengubah nilai atribut jika diperlukan. Penggunaan getter dan setter merupakan implementasi dari prinsip encapsulation dalam OOP sehingga pengelolaan data menjadi lebih aman dan terkontrol.

Pada bagian akhir terdapat method toString() yang di-override dari class Object. Method ini digunakan untuk mengubah objek mahasiswa menjadi bentuk string yang berisi nama, NIM, dan program studi dengan format tertentu. Method ini mempermudah saat objek ingin ditampilkan ke layar atau dicetak karena data mahasiswa akan otomatis ditampilkan dalam format yang lebih rapi dan mudah dibaca.

Kesimpulan : Program di atas menunjukkan sebuah class model data mahasiswa yang menerapkan konsep Object Oriented Programming (OOP) dalam Java, khususnya enkapsulasi. Class ini berfungsi sebagai tempat penyimpanan data mahasiswa yang nantinya digunakan dalam proses visualisasi dan pengurutan data pada aplikasi sorting GUI. Dengan adanya class ini, pengelolaan data mahasiswa menjadi lebih terstruktur, aman, dan mudah digunakan dalam program utama.

2. Class Sorting GUI (SortingGUI_2511533016)

```

1. package Pekan7_2511533016;
2.
3. import java.awt.BorderLayout;
4. import java.awt.Color;
5. import java.awt.Dimension;
6. import java.awt.FlowLayout;
7. import java.awt.Font;
8. import java.awt.GridLayout;
9. import java.util.ArrayList;
10.
11.     import javax.swing.BorderFactory;
12.     import javax.swing.JButton;
13.     import javax.swing.JComboBox;
14.     import javax.swing.JFrame;
15.     import javax.swing.JLabel;
16.     import javax.swing.JOptionPane;
17.     import javax.swing.JPanel;
18.     import javax.swing.JScrollPane;
19.     import javax.swing.JTextArea;
20.     import javax.swing.JTextField;
21.     import javax.swing.SwingConstants;
22.     import javax.swing.SwingUtilities;
23.
24.     public class SortingGUI_2511533016 extends JFrame {
25.
26.         private static final long serialVersionUID = 1L;
27.
28.         private JTextField txtNama_3016, txtNim_3016,
29. txtProdi_3016;
30.         private JButton addButton_3016, stepButton_3016,
31. resetButton_3016;
32.         private JComboBox<String> comboSorting_3016;
33.         private JPanel panelArray_3016;
34.         private JTextArea areaLangkah_3016;
35.         private JLabel[] labelArray_3016;
36.
37.         private ArrayList<Mahasiswa_2511533016> listMhs_3016;

```

```

36.         private ArrayList<Mahasiswa_2511533016>
           dataSorting_3016;
37.
38.         private int i_3016;
39.         private boolean sortingSelesai_3016;
40.
41.         public SortingGUI_2511533016() {
42.             setTitle("Visualisasi Sorting Mahasiswa");
43.             setSize(750, 400);
44.             setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
45.             setLocationRelativeTo(null);
46.             setLayout(new BorderLayout());
47.
48.             listMhs_3016 = new ArrayList<>();
49.
50.             // Panel Input
51.             JPanel panelInput_3016 = new JPanel(new
           GridLayout(5, 2, 5, 5));
52.             panelInput_3016.setBorder(
53.                 BorderFactory.createTitledBorder("Input
           Data"));
54.
55.             panelInput_3016.add(new JLabel("Nama Mahasiswa"));
56.             txtNama_3016 = new JTextField();
57.             panelInput_3016.add(txtNama_3016);
58.
59.             panelInput_3016.add(new JLabel("NIM"));
60.             txtNim_3016 = new JTextField();
61.             panelInput_3016.add(txtNim_3016);
62.
63.             panelInput_3016.add(new JLabel("Program Studi"));
64.             txtProdi_3016 = new JTextField();
65.             panelInput_3016.add(txtProdi_3016);
66.
67.             panelInput_3016.add(new JLabel("Pilih Sorting"));
68.             comboSorting_3016 = new JComboBox<>();
69.             comboSorting_3016.addItem("Insertion Sort");
70.             comboSorting_3016.addItem("Selection Sort");
71.             comboSorting_3016.addItem("Bubble Sort");
72.             panelInput_3016.add(comboSorting_3016);
73.
74.             addButton_3016 = new JButton("Tambah Data");
75.             panelInput_3016.add(addButton_3016);
76.
77.             // Panel Visual Array
78.             panelArray_3016 = new JPanel();
79.             panelArray_3016.setLayout(new FlowLayout());
80.             panelArray_3016.setBorder(
81.                 BorderFactory.createTitledBorder("Visualisasi Data"));
82.
83.             // Panel Control
84.             JPanel panelControl_3016 = new JPanel();

```

```

85.         stepButton_3016 = new JButton("Langkah
Selanjutnya");
86.         resetButton_3016 = new JButton("Reset");
87.         panelControl_3016.add(stepButton_3016);
88.         panelControl_3016.add(resetButton_3016);
89.
90.         // Text Area
91.         areaLangkah_3016 = new JTextArea();
92.         areaLangkah_3016.setEditable(false);
93.         areaLangkah_3016.setFont(new Font("Monospaced",
Font.PLAIN, 14));
94.         JScrollPane scroll_3016 = new
JScrollPane(areaLangkah_3016);
95.         scroll_3016.setPreferredSize(new Dimension(350,
500));
96.
97.         // Add Component
98.         add(panelInput_3016, BorderLayout.NORTH);
99.         add(panelArray_3016, BorderLayout.CENTER);
100.        add(panelControl_3016, BorderLayout.SOUTH);
101.        add(scroll_3016, BorderLayout.EAST);
102.
103.        // Button Event
104.        addButton_3016.addActionListener(e ->
tambahData_3016());
105.        stepButton_3016.addActionListener(e ->
prosesSorting_3016());
106.        resetButton_3016.addActionListener(e ->
reset_3016());
107.        stepButton_3016.setEnabled(false);
108.    }
109.
110.    // Method Tambah Data
111.    private void tambahData_3016() {
112.        String nama_3016 = txtNama_3016.getText().trim();
113.        String nim_3016 = txtNim_3016.getText().trim();
114.        String prodi_3016 = txtProdi_3016.getText().trim();
115.
116.        if (nama_3016.isEmpty() || nim_3016.isEmpty() ||
prodi_3016.isEmpty()) {
117.            JOptionPane.showMessageDialog(this, "Data belum
lengkap!");
118.            return;
119.        }
120.
121.        Mahasiswa_2511533016 mhs_3016 = new
Mahasiswa_2511533016(nama_3016, nim_3016, prodi_3016);
122.        listMhs_3016.add(mhs_3016);
123.        tampilkanVisual_3016(listMhs_3016);
124.
125.        txtNama_3016.setText("");
126.        txtNim_3016.setText("");
127.        txtProdi_3016.setText("");

```

```

128.         stepButton_3016.setEnabled(true);
129.
130.         i_3016 = 1;
131.         sortingSelesai_3016 = false;
132.         areaLangkah_3016.setText("");
133.     }
134.
135.     // Visual Tabel Array
136.     private void
137.     tampilkanVisual_3016(ArrayList<Mahasiswa_2511533016> data_3016) {
138.         panelArray_3016.removeAll();
139.         labelArray_3016 = new JLabel[data_3016.size()];
140.         for (int k_3016 = 0; k_3016 < data_3016.size();
141.             k_3016++) {
142.             labelArray_3016[k_3016] = new
143.             JLabel(data_3016.get(k_3016).getNama_3016());
144.             labelArray_3016[k_3016].setFont(new
145.             Font("Arial", Font.BOLD, 16));
146.             labelArray_3016[k_3016].setBorder(BorderFactory.createLineBorder(
147.             Color.BLACK));
148.             labelArray_3016[k_3016].setPreferredSize(new
149.             Dimension(120, 50));
150.             labelArray_3016[k_3016].setHorizontalAlignment(SwingConstants.CEN
151.             TER);
152.             panelArray_3016.add(labelArray_3016[k_3016]);
153.         }
154.         panelArray_3016.revalidate();
155.         panelArray_3016.repaint();
156.     }
157.
158.     // Proses Sorting Step
159.     private void prosesSorting_3016() {
160.         if (listMhs_3016.size() <= 1) {
161.             JOptionPane.showMessageDialog(this, "Data
162.             minimal 2!");
163.             return;
164.         }
165.
166.         if (dataSorting_3016 == null || i_3016 == 1) {
167.             dataSorting_3016 = new ArrayList<>();
168.             for (Mahasiswa_2511533016 m : listMhs_3016) {
169.                 dataSorting_3016.add(new
170.                 Mahasiswa_2511533016(
171.                     m.getNama_3016(), m.getNim_3016(),
172.                     m.getProdi_3016()));
173.             }
174.         }
175.
176.         String pilihan_3016 =
177.         comboSorting_3016.getSelectedItem().toString();
178.         if (pilihan_3016.equals("Insertion Sort")) {

```

```

169.         insertionStep_3016();
170.     } else if (pilihan_3016.equals("Selection Sort")) {
171.         selectionStep_3016();
172.     } else {
173.         bubbleStep_3016();
174.     }
175.
176.     tampilkanVisual_3016(dataSorting_3016);
177. }
178.
179. // Insertion Sort Step
180. private void insertionStep_3016() {
181.     if (i_3016 < dataSorting_3016.size()) {
182.         Mahasiswa_2511533016 key_3016 =
183.             dataSorting_3016.get(i_3016);
184.         int j_3016 = i_3016 - 1;
185.         while (j_3016 >= 0 && dataSorting_3016.get(
186.             j_3016).getNama_3016().compareToIgnoreCase(
187.                 key_3016.getNama_3016()) > 0) {
188.             dataSorting_3016.set(j_3016 + 1,
189.                 dataSorting_3016.get(j_3016));
190.             j_3016--;
191.         }
192.         dataSorting_3016.set(j_3016 + 1, key_3016);
193.         areaLangkah_3016.append(
194.             "Langkah " + i_3016 + " : " +
195.             tampilNama_3016(
196.                 dataSorting_3016) + "\n");
197.         i_3016++;
198.     } else {
199.         selesai_3016();
200.     }
201. }
202.
203. // Selection Sort Step
204. private void selectionStep_3016() {
205.     if (i_3016 < dataSorting_3016.size()) {
206.         int min_3016 = i_3016 - 1;
207.         for (int j_3016 = i_3016; j_3016 <
208.             dataSorting_3016.size(); j_3016++) {
209.             if
210.                 (dataSorting_3016.get(j_3016).getNama_3016().
211.                     compareToIgnoreCase(dataSorting_3016.get(min_3016).
212.                         getNama_3016()) < 0) {
213.                 min_3016 = j_3016;
214.             }
215.         }
216.         Mahasiswa_2511533016 temp_3016 =
217.             dataSorting_3016.get(i_3016 - 1);

```



```

214.         dataSorting_3016.set(i_3016 - 1,
    dataSorting_3016.get(min_3016));
215.         dataSorting_3016.set(min_3016, temp_3016);
216.
217.         areaLangkah_3016.append("Pass " + i_3016 + " :
    "
218.         + tampilNama_3016(dataSorting_3016) +
    "\n");
219.
220.         i_3016++;
221.     } else {
222.         selesai_3016();
223.     }
224. }
225.
226. // Bubble Sort Step
227. private void bubbleStep_3016() {
228.     if (i_3016 < dataSorting_3016.size()) {
229.         for (int j_3016 = 0; j_3016 <
    dataSorting_3016.size() - i_3016; j_3016++) {
230.             if
    (dataSorting_3016.get(j_3016).getNama_3016().compareToIgnoreCase(
231.                 dataSorting_3016.get(j_3016 +
    1).getNama_3016()) > 0) {
232.                 Mahasiswa_2511533016 temp_3016 =
    dataSorting_3016.get(j_3016);
233.                 dataSorting_3016.set(j_3016,
    dataSorting_3016.get(j_3016 + 1));
234.                 dataSorting_3016.set(j_3016 + 1,
    temp_3016);
235.             }
236.         }
237.         areaLangkah_3016.append("Pass " + i_3016 + " :
    " + tampilNama_3016(
238.             dataSorting_3016) + "\n");
239.
240.         i_3016++;
241.     } else {
242.         selesai_3016();
243.     }
244. }
245.
246. // String Array
247. private String
    tampilNama_3016(ArrayList<Mahasiswa_2511533016> data_3016) {
248.     String hasil_3016 = "[";
249.
250.     for (int i_3016 = 0; i_3016 < data_3016.size();
    i_3016++) {
251.         hasil_3016 +=
    data_3016.get(i_3016).getNama_3016();
252.         if (i_3016 != data_3016.size() - 1) {
253.             hasil_3016 += ", ";
254.         }

```

```

255.         }
256.         hasil_3016 += "]";
257.         return hasil_3016;
258.     }
259.
260.     // Sorting Selesai
261.     private void selesai_3016() {
262.         if (!sortingSelesai_3016) {
263.             sortingSelesai_3016 = true;
264.             stepButton_3016.setEnabled(false);
265.             JOptionPane.showMessageDialog(this, "Sorting
selesai!");
266.         }
267.     }
268.
269.     // Reset
270.     private void reset_3016() {
271.         listMhs_3016.clear();
272.         dataSorting_3016 = null;
273.         panelArray_3016.removeAll();
274.         panelArray_3016.revalidate();
275.         panelArray_3016.repaint();
276.
277.         areaLangkah_3016.setText("");
278.         txtNama_3016.setText("");
279.         txtNim_3016.setText("");
280.         txtProdi_3016.setText("");
281.
282.         stepButton_3016.setEnabled(false);
283.         i_3016 = 1;
284.         sortingSelesai_3016 = false;
285.     }
286.
287.     // Main
288.     public static void main(String[] args) {
289.         SwingUtilities.invokeLater(() -> {
290.             new SortingGUI_2511533016().setVisible(true);
291.         });
292.     }
293. }

```

Gambar 2

Program di atas merupakan sebuah aplikasi GUI (Graphical User Interface) berbasis Java Swing yang digunakan untuk memvisualisasikan proses pengurutan data mahasiswa menggunakan beberapa metode sorting, yaitu Insertion Sort, Selection Sort, dan Bubble Sort. Program dibuat dalam class `SortingGUI_2511533016` yang merupakan turunan dari `JFrame`, sehingga aplikasi dapat menampilkan tampilan antarmuka berupa form input, tombol, area

visualisasi data, dan area langkah proses sorting. Pada bagian awal program dilakukan import berbagai library Java Swing dan AWT yang berfungsi untuk membuat komponen tampilan seperti tombol, panel, text field, label, combo box, serta pengaturan layout dan warna.

Di dalam program terdapat beberapa komponen utama seperti JTextField untuk memasukkan nama mahasiswa, NIM, dan program studi, kemudian JButton yang digunakan untuk menambah data, menjalankan langkah sorting, dan melakukan reset data. Selain itu terdapat JComboBox yang berfungsi untuk memilih metode sorting yang akan digunakan. Data mahasiswa disimpan menggunakan ArrayList<Mahasiswa_2511533016> sehingga data dapat ditambah secara dinamis sesuai kebutuhan pengguna. Variabel i_3016 digunakan sebagai penanda langkah sorting yang sedang berjalan, sedangkan sortingSelesai_3016 digunakan untuk mengetahui apakah proses sorting telah selesai dilakukan. Constructor SortingGUI_2511533016() digunakan untuk membangun tampilan utama program. Pada bagian ini ditentukan ukuran frame, layout utama, panel input data, panel visualisasi array, panel kontrol tombol, serta text area yang digunakan untuk menampilkan langkah-langkah proses sorting. Program juga memberikan event handler pada setiap tombol menggunakan ActionListener, sehingga ketika tombol ditekan maka method tertentu akan dijalankan sesuai fungsi tombol tersebut.

Method tambahData_3016() berfungsi untuk mengambil input pengguna berupa nama, NIM, dan program studi. Program akan memeriksa apakah data sudah lengkap atau belum. Jika ada data kosong maka akan muncul pesan kesalahan menggunakan JOptionPane. Jika data valid, maka objek mahasiswa baru dibuat dan dimasukkan ke dalam ArrayList. Setelah itu data langsung divisualisasikan menggunakan method tampilkanVisual_3016().

Method tampilkanVisual_3016() digunakan untuk menampilkan data mahasiswa secara visual ke dalam panel array menggunakan JLabel. Setiap label menampilkan nama mahasiswa dengan ukuran tertentu serta diberikan border agar menyerupai bentuk array atau kotak data. Tampilan ini mempermudah pengguna untuk melihat perubahan posisi data saat proses sorting berlangsung.

Method prosesSorting_3016() merupakan pusat pengendali proses sorting.

Method ini akan mengecek metode sorting yang dipilih pada combo box, kemudian menjalankan langkah sorting sesuai pilihan pengguna. Jika pengguna memilih Insertion Sort maka method `insertionStep_3016()` akan dijalankan, jika memilih Selection Sort maka method `selectionStep_3016()` dijalankan, sedangkan Bubble Sort menggunakan method `bubbleStep_3016()`.

Pada method `insertionStep_3016()`, program melakukan pengurutan dengan cara menyisipkan data pada posisi yang tepat di bagian array yang sudah terurut. Data dibandingkan menggunakan method `compareToIgnoreCase()` agar pengurutan huruf besar dan kecil tidak berpengaruh. Setelah setiap langkah selesai, hasil sorting ditampilkan ke dalam `JTextArea`.

Method `selectionStep_3016()` bekerja dengan mencari data terkecil pada bagian array yang belum terurut, kemudian menukarnya dengan posisi awal pada pass tersebut. Sedangkan method `bubbleStep_3016()` melakukan pengurutan dengan membandingkan dua data yang berdekatan lalu menukarnya jika urutannya salah. Proses ini dilakukan berulang hingga seluruh data terurut.

Method `tampilNama_3016()` digunakan untuk mengubah isi array menjadi bentuk string agar dapat ditampilkan pada area langkah sorting. Kemudian method `selesai_3016()` digunakan untuk memberikan notifikasi bahwa proses sorting telah selesai dan menonaktifkan tombol langkah berikutnya. Terakhir, method `reset_3016()` digunakan untuk menghapus seluruh data, membersihkan tampilan, serta mengembalikan program ke kondisi awal.

Kesimpulan: Program di atas menunjukkan sebuah aplikasi visualisasi algoritma sorting berbasis GUI menggunakan Java Swing yang berfungsi untuk membantu pengguna memahami proses pengurutan data mahasiswa secara bertahap. Program ini tidak hanya melakukan pengurutan data, tetapi juga menampilkan proses perpindahan data pada setiap langkah sehingga mempermudah pembelajaran konsep algoritma Insertion Sort, Selection Sort, dan Bubble Sort secara interaktif dan visual.

OUTPUT :

MENAMBAHKAN MAHASISWA :

SORTING INSERTION SORT :

SORTING SELECTION SORT :

SORTING BUBBLE SORT :